Introduction to Python

A beginner-friendly guide to your first steps in programming

1. What is Python?

Python is a popular, high-level programming language known for its readable syntax. It is used in many fields, including web development, automation, data analysis, artificial intelligence, scientific computing, and education. Because Python code often looks close to everyday language, it is a welcoming first language for new programmers.

Why learn Python?

- **Easy to read:** clear syntax helps you focus on problem-solving.
- **Versatile:** one language can support many kinds of projects.
- **Large community:** tutorials, documentation, and helpful libraries are widely available.
- **Quick to experiment:** you can write a small program and see the result immediately.

2. Your first Python program

A Python program is a set of instructions. The classic first example prints a message on the screen:

```
print("Hello, world!")
```

The **print()** function displays information. Text is written inside quotation marks, and the parentheses tell Python what to print.

3. Basic building blocks

Variables store values so you can use them later. Python determines the type automatically:

```
name = "Maya"
age = 18
print(name)
print(age)
```

Common data types include strings (`str`) for text, integers (`int`) for whole numbers, floats (`float`) for decimal numbers, and booleans (`bool`) for True or False values.

Operators let you calculate and compare values. For example, + adds, * multiplies, == checks equality, and >= checks whether one value is greater than or equal to another.

4. Making decisions and repeating tasks

An **if** statement lets a program choose what to do. Indentation (the spaces at the start of a line) is important in Python because it groups instructions together.

```
temperature = 25
if temperature > 30:
    print("It is hot.")
else:
    print("It is not too hot.")
```

Loops repeat instructions. A **for** loop is useful when working through a sequence:

```
for number in range(1, 4):
    print(number)
```

This prints 1, 2, and 3. The `range(1, 4)` function stops before 4.

5. Functions: reusable pieces of code

A function groups instructions under a name. You can call it whenever you need that task. Functions help keep programs organized and reduce repeated code.

```
def greet(person):
    print("Hello, " + person + "!")

greet("Sam")
```

Here, `person` is a parameter: it receives the value supplied when the function is called. The example displays “Hello, Sam!”.

6. Working with collections

A **list** stores multiple items in a particular order. Lists are useful when you want to keep related values together.

```
fruits = ["apple", "banana", "orange"]
print(fruits[0])
fruits.append("grape")
```

List positions start at **0**, so `fruits[0]` refers to the first item. The `append()` method adds an item to the end.

7. How to start practicing

1. Install Python from the official website, or use an online Python editor to begin without setup.
2. Write and run small examples; change values and observe what happens.
3. Practice one concept at a time: variables, conditions, loops, and functions.
4. Build a tiny project, such as a calculator, quiz, or number-guessing game.
5. When an error appears, read the message carefully and test one fix at a time.

Key idea

Programming is a skill learned through practice. Start with small problems, experiment, and build on what you already understand. Python's official documentation and beginner tutorials are useful places to continue learning.

Quick glossary

Term	Meaning
Code	Instructions written for a computer to follow.
Syntax	The rules for writing valid Python statements.
Bug	A mistake that causes a program to behave incorrectly.
Library	Reusable code that adds functionality to a project.

Learn more: **python.org** — the official home of Python.